



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN



机器学习

实验二：基于 PyTorch 的手写数字识别

2026年春

目录

CONTENTS

1

实验目的

2

PyTorch环境搭建

3

知识点回顾

4

Pytorch实例应用

5

实验任务

6

实验提交

一、实验目的

- 理解 BP 神经网络基本原理，掌握其核心工作机制
- 掌握使用 PyTorch 搭建全连接网络实现 MNIST 手写数字识别
- 掌握深度学习实验基本流程，包括数据预处理、迭代训练及评估等关键环节
- 通过多组消融实验，探究不同超参数对模型性能的影响
- 能够通过训练曲线分析模型状态，独立完成多组对比实验

二、PyTorch环境搭建 (1)

➤ 创建独立虚拟环境

01

创建新环境

为避免版本冲突，创建一个干净的隔离环境：

```
conda create -n ml_lab2  
python=3.10
```

创建名为 ml_lab2 的环境

02

激活环境

查当前终端中使用的Python版本是否正确：

```
conda activate ml_lab2
```

激活 ml_lab2 环境

03

验证环境

查当前终端中使用的Python版本是否正确：

```
python -V
```

输出 Python 3.10.x 即成功



提示：为 PyTorch 创建独立虚拟环境，避免与现有环境产生依赖冲突，保持系统整洁。若之前已创建过类似的 conda 环境（如实验1环境），可直接使用 `conda activate <环境名>` 激活，并在此基础上加装 PyTorch，无需重复创建。

二、PyTorch环境搭建 (2)

➤ 安装 PyTorch CPU版本



01 安装 PyTorch CPU

确保已在终端激活虚拟环境 **ml_lab2**，然后执行下方命令，指定 CPU 版本的下载源进行安装。

```
pip install torch torchvision  
torchaudio --index-url  
https://download.pytorch.org/whl/cpu
```



02 验证安装结果

安装完成后，在激活的环境中执行下方命令。若终端成功打印出版版本号、安装位置等详细信息，则表示安装无误。

```
pip show torch
```

> 显示 Version, Location, Requires 等字段信息即为成功

二、PyTorch环境搭建 (3)

➤ 安装 PyTorch GPU版本 (可选)



三大核心依赖组件



NVIDIA 显卡驱动 (Driver)

版本需足够新，以完整支持后续安装的 CUDA 版本。



CUDA 工具包 (Toolkit)

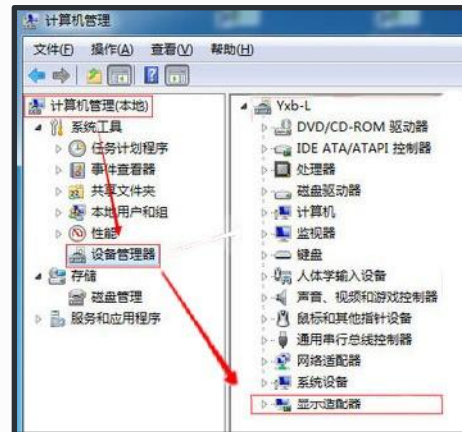
NVIDIA推出的并行计算架构，是实现通用 GPU加速的底层基础。



cuDNN SDK (GPU 加速库)

专为深度神经网络运算设计的GPU加速库，大幅提升训练速度。

如何确认你的电脑有GPU?



检查步骤:

1. 右键点击“此电脑” → 选择“管理”。
2. 打开左侧“设备管理器” → 点击“显示适配器”。
3. 查看列表中是否有 NVIDIA 显卡名称。

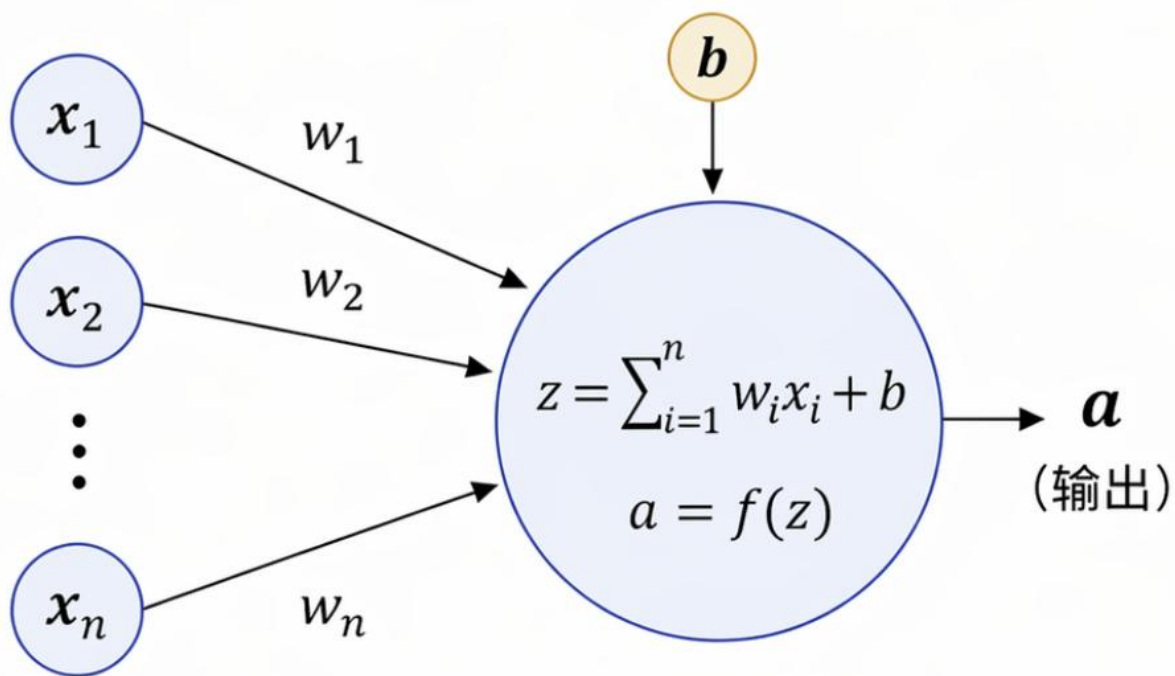


重要提示:

若有 NVIDIA 显卡，可安装 GPU 版本，以获得 5-10 倍的训练加速。若无，CPU 版本也能满足基础的学习和实验需求。

三、知识点回顾 ——BP神经网络 (1)

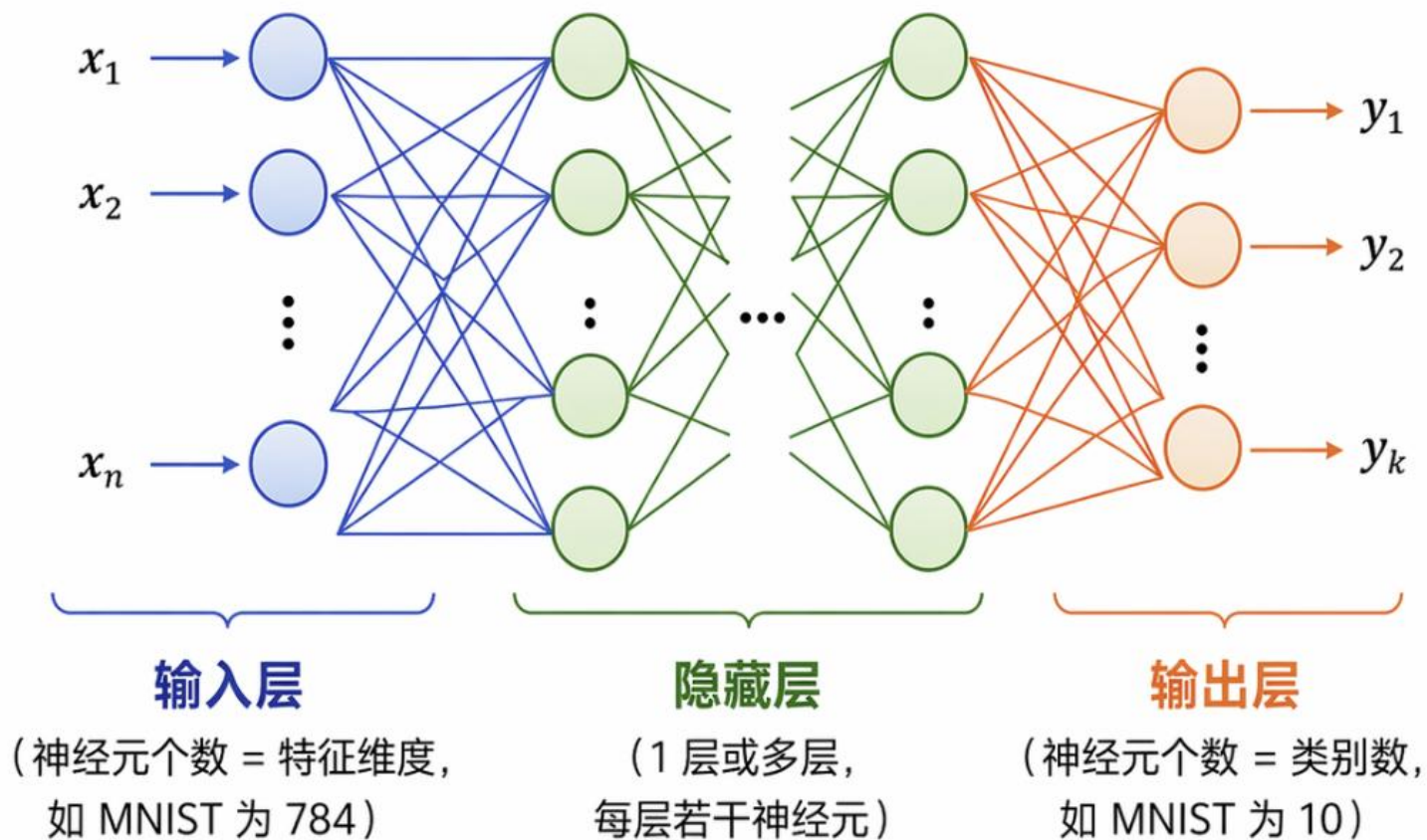
1 神经元模型



- 输入: $x_1, x_2, \dots, x_n$
- 权重: $w_1, w_2, \dots, w_n$
- 偏置: b
- 加权和: $z = \sum_{i=1}^n w_i x_i + b$
- 输出: $a = f(z)$

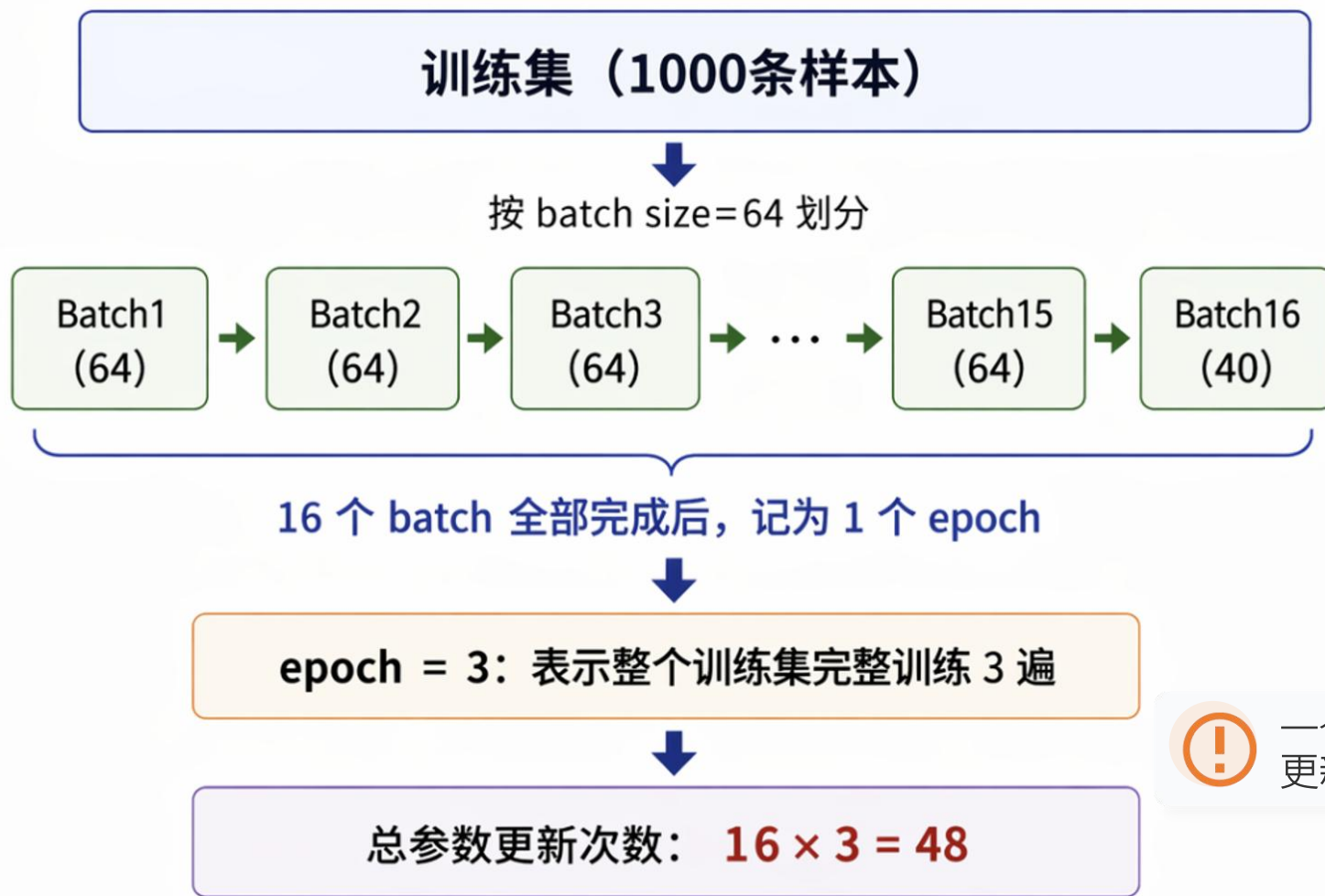
二、知识点回顾——BP神经网络 (2)

2 BP神经网络基本结构



三、知识点回顾 ——BP神经网络 (3)

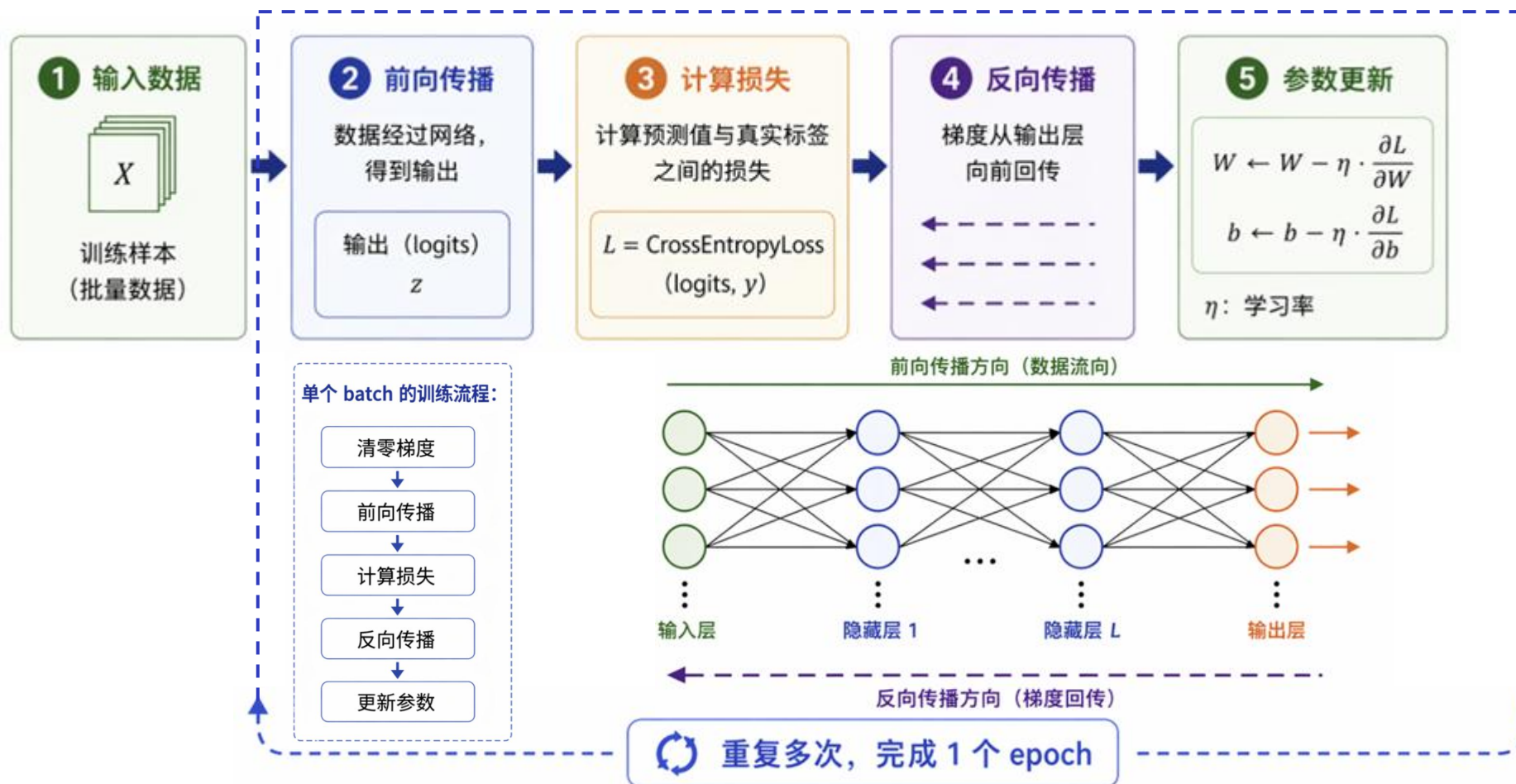
3 神经网络训练流程



一个batch
更新一次参数!

三、知识点回顾——BP神经网络（4）

3 神经网络训练流程

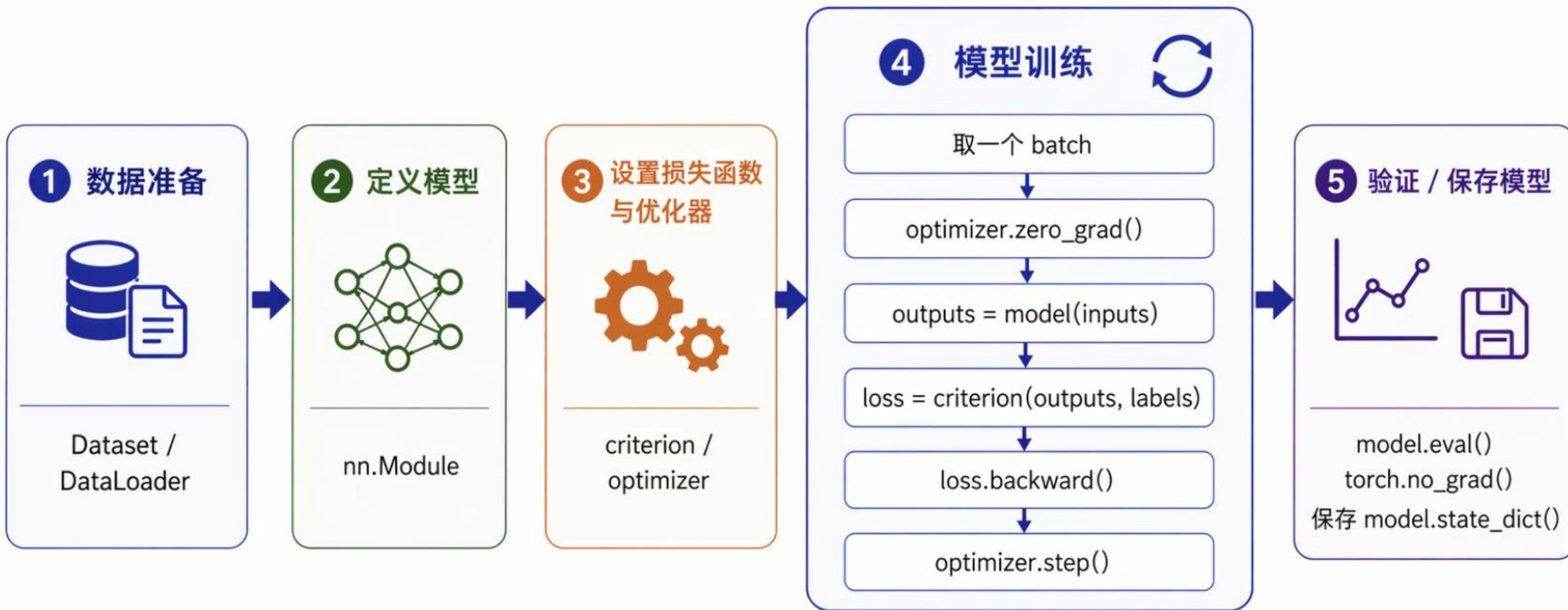


三、知识点回顾 —— 主流深度学习框架对比

框架	开发主体	核心定位与特点	易用性
PyTorch	Meta (Facebook)	研究优先，灵活直观。以动态图（Eager Execution）起家，拥有非常Pythonic的API设计，调试简单，是 学术界和研究中最受欢迎的框架 。	☆☆☆☆☆
TensorFlow	Google	生产优先，全面强大。以静态图起家，拥有强大的生产端部署工具链（TFX, TFLite, TF.js等），生态极其庞大，是 企业级应用的传统强者 。	☆☆☆
PaddlePaddle	百度	产业实践，中文友好。强调与产业结合，内置了众多经过实践验证的模型库（如PaddleOCR、PaddleClas）。 中文文档和社区支持是其最大优势 。	☆☆☆☆
MindSpore	华为	全场景协同，性能优化。主打“ 端-边-云 ”全场景协同部署，与华为昇腾AI处理器（Ascend）深度绑定，追求极致的性能。	☆☆☆

四、PyTorch实例应用

➤ 本实验代码主线：从数据到结果



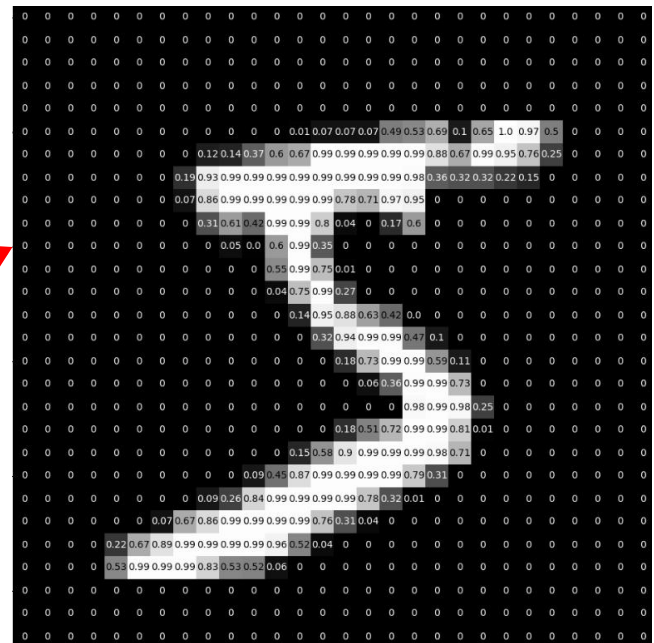
四、PyTorch实例应用

➤ 手写数字识别

MNIST 数据集 是深度学习入门的首选实践数据集。由70,000张手写数字的灰度图像组成，每张图像都是经过尺寸归一化和中心对齐处理的 28×28 像素的灰度图，并配有对应的数字标签（0-9）。

- 训练集：包含60,000张图像，用于模型训练
- 测试集：包含10,000张图像，用于评估模型性能

任务类型：10分类问题



(1) 数据预处理与数据划分

- **ToTensor()**: 将 0~255 像素值转换为 0~1 的张量
- **Normalize()**: 使用 MNIST 均值和标准差进行标准化

- train: 用于训练模型参数
- val: 用于调参和绘制曲线
- **test: 仅用于最终模型评估**

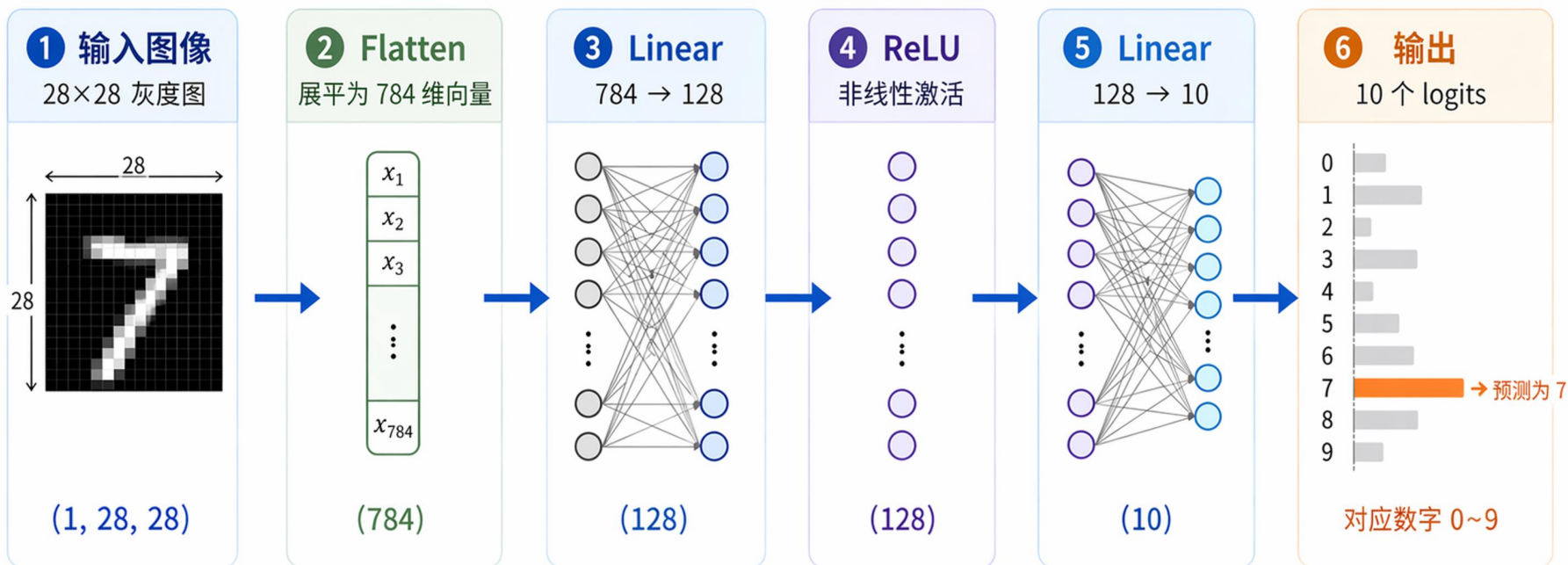
1

[illegible]

四、PyTorch实例应用

❖ (2) 定义模型：基线模型与 `nn.Sequential()`

`nn.Sequential()` 是一个顺序容器：它可以把多个网络层按前后顺序连接起来，输入会依次经过这些层，得到最终输出。



基线模型

```
model = nn.Sequential(  
    nn.Flatten(),  
    nn.Linear(28*28, 128),  
    nn.ReLU(),  
    nn.Linear(128, 10)  
)
```

四、PyTorch实例应用

(3) 定义模型：常用网络层与扩展层

类别	PyTorch 写法	作用
展平层	<code>nn.Flatten()</code>	将 28×28 图像展平成 784 维向量
全连接层	<code>nn.Linear(in_features, out_features)</code>	输入特征到输出特征的线性变换
激活函数	<code>nn.ReLU()</code> / <code>nn.Sigmoid()</code> / <code>nn.Tanh()</code> / <code>nn.LeakyReLU()</code>	为网络加入非线性表达能力
Dropout	<code>nn.Dropout(p)</code>	训练时随机丢弃部分神经元，减轻过拟合
BatchNorm	<code>nn.BatchNorm1d(num_features)</code>	可选，适合全连接层的一维特征



若完成附加题中的自定义网络，可进一步尝试 `nn.Conv2d()`、`nn.MaxPool2d()`、`nn.AvgPool2d()`、`nn.BatchNorm2d()` 等层。

四、PyTorch实例应用



(4) 训练、验证与最终测试



训练模式 `model.train()`

- ① 设置为训练模式
- ② 遍历 `train_loader`
- ③ 前向传播, 计算 `loss`
- ④ 反向传播, 更新参数
- ⑤ 记录 `train loss / train acc`



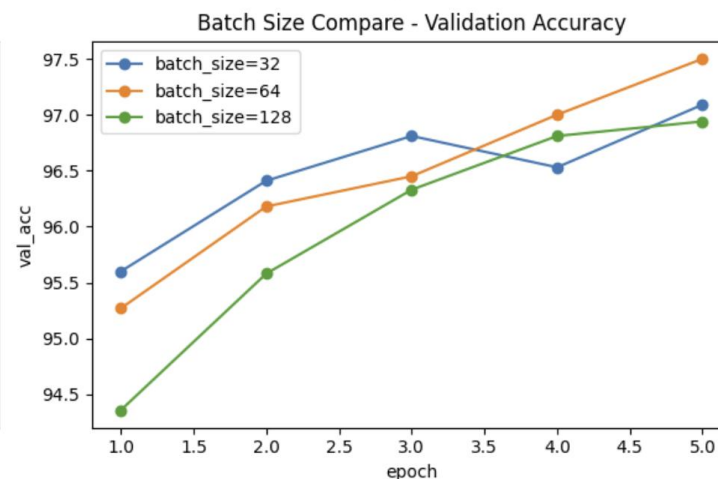
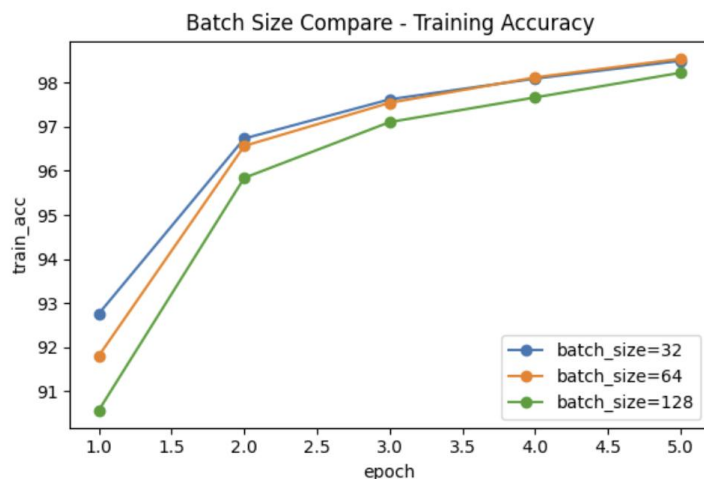
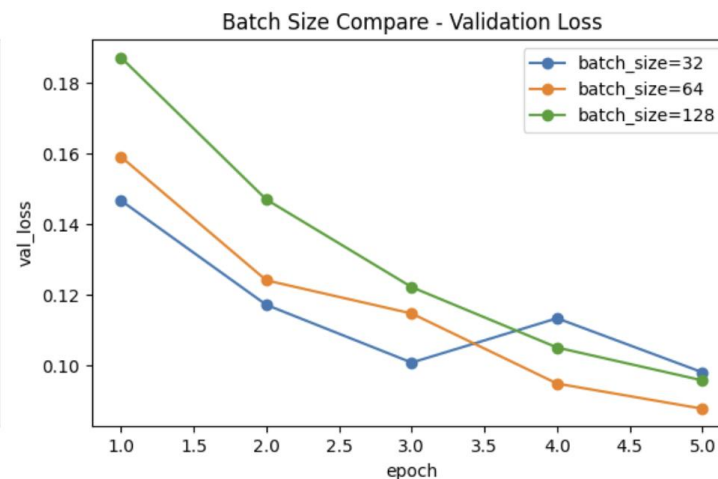
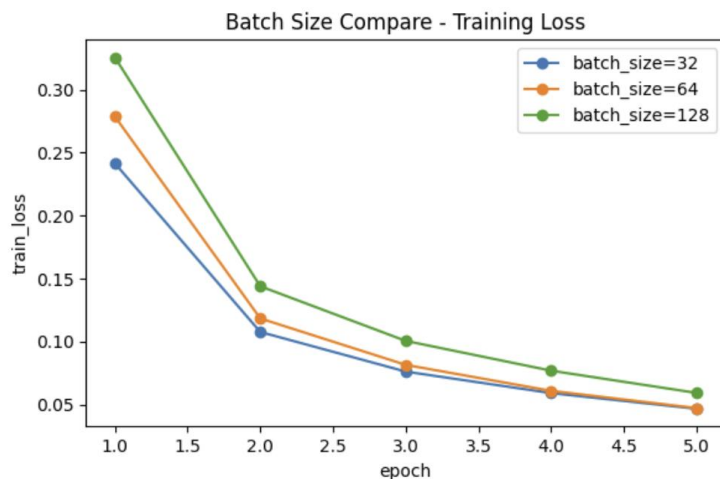
验证 / 最终测试 `model.eval()`

- ① 调参阶段使用 `val_loader`
- ② 配合 `torch.no_grad()` 关闭梯度
- ③ 记录 `val loss / val acc`, 并绘制曲线
- ④ 最终模型只在 `test_loader` 上评估一次

四、PyTorch实例应用



(5) 运行结果与观察指标 (1)



四、PyTorch实例应用



(5) 运行结果与观察指标 (2)

情况	曲线表现	说明	可以怎么做
欠拟合	<code>train_loss</code> 、 <code>val_loss</code> 都较高; <code>train_acc</code> 、 <code>val_acc</code> 都较低	模型还没学好	- 增加训练轮数 - 增大模型规模 - 调整学习率 - 适当减小 dropout
过拟合	<code>train_loss</code> 继续下降、 <code>train_acc</code> 继续上升; 但 <code>val_loss</code> 上升或 <code>val_acc</code> 停止提升	模型对训练集学得太“死”	- 提前停止训练 - 增加 dropout - 加入正则化 - 做数据增强 - 适当减小模型规模
可以继续训练	<code>val_loss</code> 还在下降; <code>val_acc</code> 还在上升	模型还有提升空间	- 继续训练 - 继续观察验证集变化
可以考虑停止	<code>val_loss</code> 不再下降; <code>val_acc</code> 不再提升	模型基本达到当前效果上限	- 停止训练 - 保留验证集表现最好的模型
比较不同参数	<code>val_acc</code> 更高; <code>val_loss</code> 更低; 曲线更平稳	调参阶段主要看验证集表现	- 选择验证集效果更好的参数设置

四、PyTorch实例应用



(6) 单变量调参：本实验主要修改哪些参数



做单变量对比实验时，每次只改变一个因素，其余参数保持与基线一致。

调参项	典型修改	本实验取值 / 说明
batch size	修改 DataLoader 中的 batch_size	32 / 64 / 128
网络深度	增减 “Linear + 激活函数” 组合	1 / 3 / 5 层（隐藏层层数）
神经元数量	修改隐藏层宽度	64 / 128 / 256 / 512
激活函数	替换为 ReLU / Sigmoid / Tanh / LeakyReLU	其余参数保持不变
Dropout	在激活函数后加入 nn.Dropout(p)	p = 0.2 或 0.5

四、PyTorch实例应用



(7) 训练配置补充：优化器 (Optimizer)

优化器： 负责根据梯度更新模型参数，会影响模型收敛速度和稳定性。

优化器	常用写法	特点 / 说明
Adam	<code>optim.Adam(model.parameters(), lr=0.001)</code>	自适应学习率，收敛较快；本实验基线推荐使用
SGD + momentum	<code>optim.SGD(model.parameters(), lr=0.01, momentum=0.9)</code>	经典优化方法；通常需要更仔细地调整学习率
RMSprop	<code>optim.RMSprop(model.parameters(), lr=0.001)</code>	自适应优化方法，可作为扩展了解
Adagrad	<code>optim.Adagrad(model.parameters(), lr=0.01)</code>	会累积历史梯度，学习率逐渐变小

四、PyTorch实例应用

(8) 训练配置补充：损失函数（Loss Function）

损失函数： 用于衡量预测结果与真实标签的差异，是反向传播和参数更新的依据。

损失函数	适用任务	注意事项
CrossEntropyLoss()	多分类问题	本实验使用；输入为 logits，标签为类别索引；模型最后不要加 Softmax。本实验基线推荐使用
NLLLoss()	多分类问题	输入应为 log-probabilities，通常与 LogSoftmax 配合使用
BCELoss()	二分类问题	输入通常需要先经过 Sigmoid，表示概率
BCEWithLogitsLoss()	二分类问题	输入为 logits，内部已包含 Sigmoid，数值更稳定
MSELoss() / L1Loss()	回归问题	用于预测连续数值，不适合本实验的 10 类分类任务

五、实验任务

任务一：环境搭建与基线运行

- ① 安装或使用 PyTorch 环境 (CPU/GPU 均可)
- ② 运行教师下发的 .ipynb, 完成基线模型训练
- ③ 截图提交:
 - 终端环境信息
 - Jupyter 内核选择
 - 最后 3 个 epoch 的 train/val 指标
 - 最终 test 结果

五、实验任务

任务二：单变量调参与对比实验

➤ 调参维度与测试取值：

- ① batch size: 32 / 64 / 128
- ② 隐藏层层数: 1 / 3 / 5
- ③ 神经元数量: 64 / 128 / 256 / 512
- ④ 激活函数: ReLU / Sigmoid / Tanh / LeakyReLU
- ⑤ Dropout: $p = 0 / 0.2 / 0.5$

➤ 实验要求：

- 每类实验均需完成，分别绘制 train/val loss 与 train/val acc 曲线；
- 汇总 “不同调参方向的最佳单变量结果对比” ；
- 最终仅在测试集上完成 1 次客观评估。

五、实验任务

➤ 附加题（选做）

完成以下任意题目可获得额外加分：

(1) 联合调参探索：在单变量实验基础上，同时调整多个参数，并与基线或最佳单变量结果进行对比。

(2) 数据增强：仅对训练集使用 `transforms.RandomRotation`、`RandomCrop` 等图像变化操作，观察验证/测试结果变化。

(3) 自定义网络结构：设计不同于基础全连接 BP 网络的模型，如加入 `BatchNorm`、`Conv2d`、`MaxPool2d`、残差连接等，并与基线模型对比。



注意：各类单变量实验中分别选出的最佳设置，直接组合后不一定仍然最优；本题重在探索和分析。每完成一题酌情加 2 ~ 4 分，全部正确完成加 10 分。本次实验总分不超过 100 分。

六、实验提交

- 作业提交平台: grader.hitsz.edu.cn/courses
- 截止时间:
 - 第一组: 5 月 27 日晚 23:59
 - 第二组: 5 月 30 日晚 23:59
 - 补交按得分80%计算分数
- 压缩包命名: 学号_姓名_机器学习实验二.zip
- 提交内容: 实验报告、代码文件 (.ipynb) , 按框架代码要求完成